Defensive Design

Introduction

When designing programs we need to ensure that they are robust. Failure to do this may result in two critical outcomes:

- The program won't function correctly / will not do the job it's supposed to do.
- The program may be unsecure and data that it holds may become compromised.

Both outcomes are unacceptable. Luckily there are ways that we can minimise these issues, through defensive design strategies, which will ensure the robustness of our software solutions:

Input Sanitisation

Anticipating Misuse and Input Validation

Authentication

Maintainability of Code

Testing

Input Sanitisation and Validation

To interact with a program, users will often input some data and it is at this point that a program's functionality and security could be compromised. If a program receives unexpected inputs, it needs to be able to deal with them appropriately so that it can continue to function. As a result, there are methods that can be used to prevent users from entering data that the program doesn't want:

Input Sanitisation

Input Validation

Input Sanitisation

This is where data entered is cleaned of any unwanted characters that the user may enter. For example, if a user is entering a password, it may be sensible to strip out any spaces, to avoid the potential security threat of an SQL injection attack.

...what is an SQL injection??

SQL Injections

SQL stands for Structured Query Language and is used to lookup data in a database. When you log into an account, you will add your username and password into a couple of input boxes. When you press 'enter', in the background these two pieces of information are added to an SQL statement:

E.g: SELECT account WHERE username = "bjones" AND password = "pa$$w0rd"

With SQL injections, you can 'bolt on' some SQL to the end of your password. This will then alter the SQL statement and allow you to access the accounts of other users. Using input

Computer Science UK Membership Site Licence: Do not share outside your centre.

sanitisation we can remove certain characters which could enable the user to misuse the system. Input validation can also help in this situation.

Input Validation

Input validation is when a system will check that the input meets certain criteria, to ensure that the data is in an acceptable form. In the situation where a user signs up to a user account, if he/she inputs their email address, input validation can check to see if the entered email address is in the expected format (contains an @ symbol and ends with a domain type (.co.uk)). If the entered address doesn't contain these items then it can be rejected in order to ensure that only valid data is entered by the user.

Deciding on what to sanitise and validate...

There are of course an overwhelming number of different inputs that users could enter into a system and to plan against every one can be a very difficult process. Software developers will often use whitelists or blacklists to help them plan against unwanted inputs.

- A whitelist is a list of data that the program being created, should accept. All other data should be rejected by the program.
- A blacklist is a list of data that the program being created, should reject. All other data should be accepted by the program.

Which do you think will be the most effective at preventing unwanted inputs and why?

Whitelist VS Blacklist





Whitelists are often more effective and secure. They are much more restrictive and often take a long time to produce but at least you know that the program cannot accept any data that you have not already planned for.

On the other hand, blacklists are often quite quick to produce and do not make the system as restrictive, but it is often impossible to think of every input that could cause threat to the system and are therefore not as secure.

Authentication

Another defensive strategy is to ensure that systems have some form of authentication. In other word, having passwords to only allow certain users and potentially limiting the access of the various parts of the system (access rights). Security can be increased further by ensuring that users update their passwords regularly, have complex passwords and limit login attempts.

Computer Science UK Membership Site Licence: Do not share outside your centre.

Maintainability

Another defence strategy is to ensure that code is maintainable. This means that code is written in such a way that when other programmers are asked to develop the code, they can make full sense of it, therefore reducing the chance of introducing coding mistakes / bugs.

To make source code more maintainable, programmers should write their code with the following features:

Comments

Indentation

Formatting

Comments

Most languages will allow you to write text amongst your code which is ignored by the compiler. This facility is used as 'comments' which are important in making the code understandable for both the programmer or their colleagues. They should be written during the coding process – not after. The following code should be included:

Program Header: At the beginning of the code there should be the program name, author, date of last edit, program description, bug fixes & remaining issues.

Subroutines: Each one should have a description or its purpose, how it is called, parameters that are passed in and values that are returned.

Constant & Variable Declaration: These should have a short description of their purpose.

Throughout Main Code: There should be sufficient detail about what is happening throughout the algorithm. They should appear before the line / block of code. If the code was deleted, the comments should be able to enable the programmer to rewrite it.

Indentation

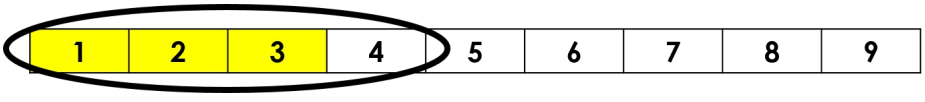
The use of indentation is extremely important when coding so that code is readable and easier to follow. Each time a new code structure is used, the code within the structure should be indented underneath. This should create a strip of whitespace down the page from the beginning to the end of the structure. This makes it clear where each structure starts and finishes and which structures are nested within others. Although Python forces indentation as part of the language, most do not and so it is important that the programmer implements this correctly themselves.

Formatting

As well as indentation, the code should be grouped together in logical blocks by splitting them up with blank lines. Think of this as splitting your code up like you would an essay with paragraphs.

Questions *(The question zone you choose must either match your target grade or be higher!)*

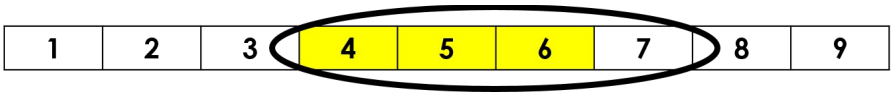
Question Zone 1-3



Computer Science UK Membership Site Licence: Do not share outside your centre.

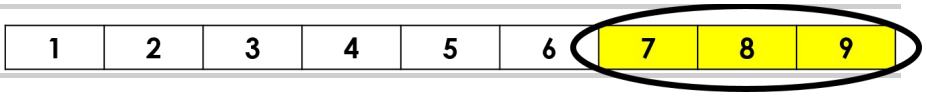
1. What is meant by the term 'Validation'? [2]
2. What is meant by the term 'Authentication' and how can it be used to make a program more secure? [3]
3. State and describe 3 methods of writing maintainable code. [3]

Question Zone 4-6



1. State and describe 3 methods of writing maintainable code. [3]
2. What is meant by the term 'Input Sanitisation' and how can it prevent SQL injections. Explain your answer in full. [3]
3. Explain the differences between input sanitation and input validation [4]

Questions Zone 7-9



1. Explain the differences between input sanitation and input validation [4]
2. Discuss the advantages and disadvantages of whitelists and blacklists. [6]
3. Discuss how the maintainability of code can lead to robust programs. [4]

Computer Science UK Membership Site Licence: Do not share outside your centre.

Checklist:

- Date and title, clearly presented
- Spelling & grammar checked
- Question numbers in the margin
- Handwriting neat & legible
- Punctuation / Capital letters

Keywords / Key Terms:

Defensive Design: The application of a range of strategies, when designing/writing programs, to ensure that they are robust (error free and meet the needs of the end user).

Input Sanitisation: Inputted data entered is cleaned of any unwanted characters.

Input Validation*:* Checking that the input meets certain criteria, to ensure that the data is in an acceptable form.

Computer Science UK Membership Site Licence: Do not share outside your centre.

State/Identify/Give/Name: Simply label a diagram, fill out a table or write a few words
Describe: Describing is 'saying what you see' (E.G.: A computer will have a CPU, Primary and Secondary storage etc)
Explain: Explaining is 'saying WHY/HOW something is like that' (E.G.: A computer will have a CPU so that it can process all of the data the computer needs to perform a range of tasks. Primary and Secondary storage is needed because...)
Discuss: Discussing is 'looking at two sides of an issue, weighing up the two views and giving a conclusion'. Often these require a mini essay answer. (E.G.: New technology could be seen as being bad for the environment because..., but on the other hand, new technology has led to... In conclusion I believe that...)
Describe/Explain/Discuss **using examples**: Finally, if you are asked to give examples in any of these types of questions – YOU MUST **GIVE EXAMPLES**!

Stick answer sheet here

Computer Science UK Membership Site Licence: Do not share outside your centre.

Reflections **Score:** / **Percentage:** % **Grade:** **Progress** On / Above / Below

What Went

- I demonstrated a good level of understanding.
- I responded to the command words effectively.
- My answers were detailed / were written in depth.
- My work was well presented / legible.
- My answers effectively incorporated technical terminology.
- My responses were well structured / organised.
- My revision strategy was effective as I showed depth of understanding in my answers.
- My answers contained enough points / examples / explanations to achieve the marks available.

Even Better

- My answers need to be more accurate.
- I must respond correctly to the command words.
- My answers need more detail / greater depth.
- I must take greater care over my work / write neatly.
- I must incorporate key terminology into my answers.
- I must better organise my answers to improve its clarity.
- I need to improve my revision strategy, as I did not demonstrate a depth of understanding in my answers.
- My answers didn't contain enough points / examples / explanations to achieve the marks available.

Further thoughts:

_____

_____

_____

_____

Computer Science UK Membership Site Licence: Do not share outside your centre.